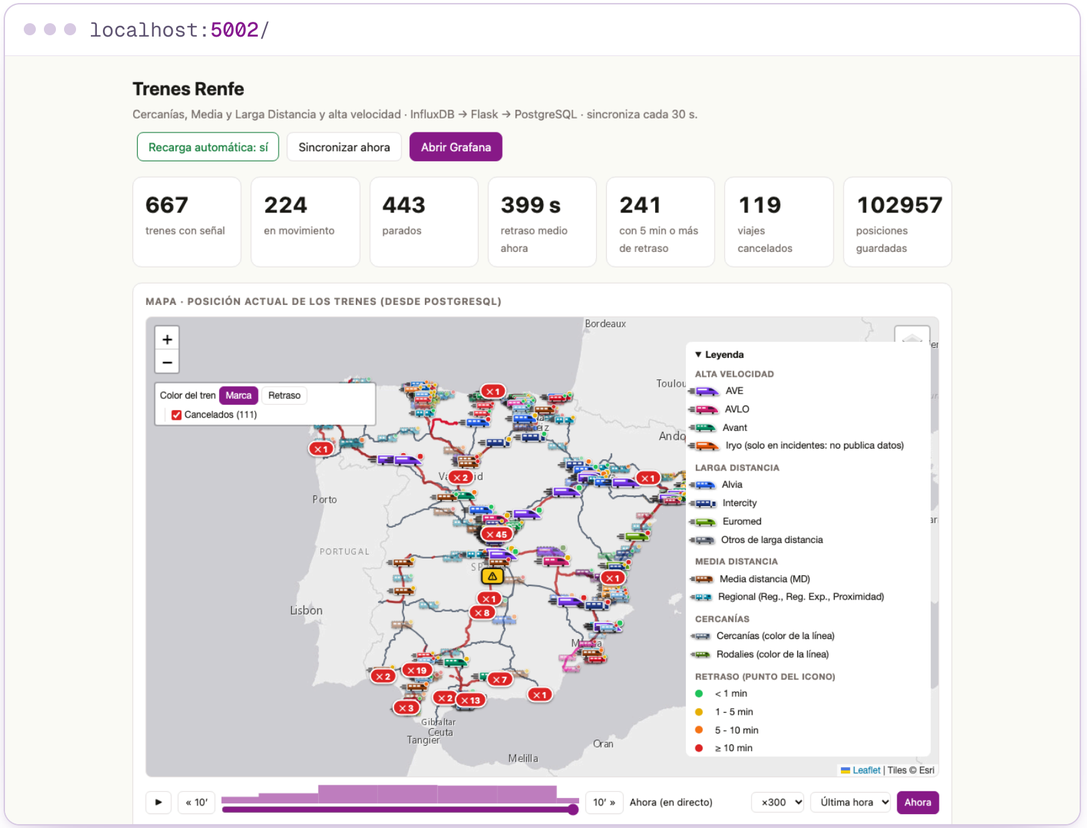
Trenes de Renfe en tiempo real

Cada tren de Renfe en un mapa vivo, con su retraso y su historial de 48 horas. Los datos abiertos de Renfe viajan por Mosquitto, Telegraf e InfluxDB hasta PostgreSQL, Flask y Grafana, todo con Docker Compose.

Práctica 3 · InfluxDB + PostgreSQL + Flask + Grafana



Un mapa de España con cada tren de Renfe, su retraso y su pasado

Parte de la práctica 1 (Mosquitto → Telegraf → InfluxDB → Grafana) y cambia la fuente por los feeds GTFS-Realtime que Renfe publica sin clave.

Se añade PostgreSQL con datos agregados y relacionados, una aplicación Flask que los mantiene y los muestra en un mapa, y gráficas nuevas en el mismo dashboard de Grafana.

Qué recoge

Posiciones y retrasos de Cercanías, Rodalies, AVE, Avant, Alvia, Intercity, Media Distancia y Regional (unos 400 + 220 trenes).

Qué guarda

Serie temporal en bruto en InfluxDB y un modelo relacional con 48 h de historial en PostgreSQL.

Qué enseña

Mapa Leaflet con un icono por marca, línea de tiempo para volver atrás, cancelados e incidentes, y dashboard de Grafana.

Del feed de Renfe a la pantalla, en ocho piezas



Cada 30 s
trenes.py baja los feeds y solo publica las posiciones que han cambiado.

MQTT a mano
Cliente MQTT propio, sin librerías, sobre un socket.

Dos bases
InfluxDB para lo reciente; PostgreSQL para relacionar y agregar.

Dos paneles
Flask para el mapa; Grafana para las gráficas.

Siete fuentes abiertas: cinco de Renfe y dos mapas

Fuente	Dirección	Para qué se usa
Posiciones Cercanías	`gtfsrt.renfe.com/vehicle_positions.json`	Latitud, longitud, estado y parada de cada tren (~400)
Retrasos Cercanías	`…/trip_updates.json`	Retraso en segundos por viaje y viajes cancelados
Posiciones Larga Distancia	`…/vehicle_positions_LD.json`	AVE, AVLO, Avant, Alvia, Intercity, MD, Regional (~220)
Retrasos Larga Distancia	`…/trip_updates_LD.json`	Retraso por viaje y cancelados de larga distancia
GTFS estático Cercanías	`ssl.renfe.com/ftransit/…/fomento_transit.zip`	Núcleo, línea, paradas y trazado de las líneas
GTFS estático LD	`ssl.renfe.com/gtransit/…/google_transit.zip`	Tipo de servicio por número de tren, paradas, origen y destino
Red ferroviaria	OpenStreetMap · Overpass	Trazado de las vías principales y de alta velocidad

Ocho contenedores que arrancan con un solo comando

Servicio	Puerto	Acceso	Función
Grafana	`3001`	admin / admin	Dashboard «Trenes Renfe» con dos fuentes de datos
Flask	`5002`	sin login	Web con mapa y API: `/api/instante`, `/sync`, `/health`…
PostgreSQL 16	`5433`	practica3 / practica3	Modelo relacional, se inicializa con `init.sql`
InfluxDB 1.8	`8086`	root / root	Base `renfe` con autenticación activada
Mosquitto 2	`1883`	anónimo	Broker MQTT entre trenes.py y Telegraf
Telegraf	-	-	Cuatro entradas MQTT hacia InfluxDB
pgAdmin 4	`5050`	sin login	Visor de PostgreSQL con el servidor ya registrado
Chronograf	`8888`	sin login	Visor de InfluxDB (*Explore*) ya conectado

InfluxDB guarda lo que llega; PostgreSQL lo ordena y lo relaciona

InfluxDB · serie temporal

Telegraf escribe cada mensaje MQTT tal como llega, con etiquetas para filtrar rápido.

- **trenes**: posición de cada tren. Etiquetas `tren, linea, nucleo, categoria`.
- **trenes_nucleo**: trenes activos y retraso medio por núcleo.
- **trenes_stats**: contadores globales (activos, retraso medio, con retraso).
- **trenes_cancelados**: viajes con `CANCELED`, origen y destino.

`tiempo real` `Grafana` `Chronograf`

PostgreSQL · modelo relacional

Flask copia lo nuevo cada 30 s, lo normaliza en dimensiones y calcula agregados con SQL.

- **Dimensiones**: nucleos, lineas, trenes, paradas.
- **Hechos**: posiciones, con 48 h de historial.
- **Agregados**: retrasos_hora y retrasos_dia.
- **Vistas y función**: trenes_en(instante), v_lineas, v_nucleos…

`histórico` `joins` `línea de tiempo`

Un esquema de dimensiones, hechos y agregados

Tabla o vista	Contenido
`nucleos` · `lineas` · `trenes` · `paradas`	Dimensiones. Núcleo (Madrid, Rodalies…), línea (C3, AVE…), cada tren dentro de su línea y las paradas del GTFS.
`posiciones`	Hechos. Cada posición con velocidad estimada, retraso y parada. Se conservan 48 h.
`retrasos_hora` → `retrasos_dia`	Agregados: por hora y línea desde `posiciones`; por día y núcleo desde `retrasos_hora`.
`cancelaciones` · `v_cancelados_ahora`	Viajes cancelados con origen y destino; la vista muestra los de los últimos 15 min.
`incidentes`	Sucesos escritos a mano: posición, trenes implicados, texto y fuentes.
`trenes_en(instante)` · `v_trenes_ahora`	Última posición de cada tren en un instante, con `en_movimiento`. La usa la línea de tiempo.
`v_lineas` · `v_nucleos` · `sincronizaciones`	Retraso medio y % de puntualidad en la última hora; registro de cada pasada de Flask.

Un icono distinto para cada marca de tren

Vías y marcas

Red principal en gris, alta velocidad en rojo y líneas de Cercanías con su color. AVE violeta, AVLO rosa, Avant esmeralda, Alvia azul, Intercity marino, MD marrón.

Color del tren

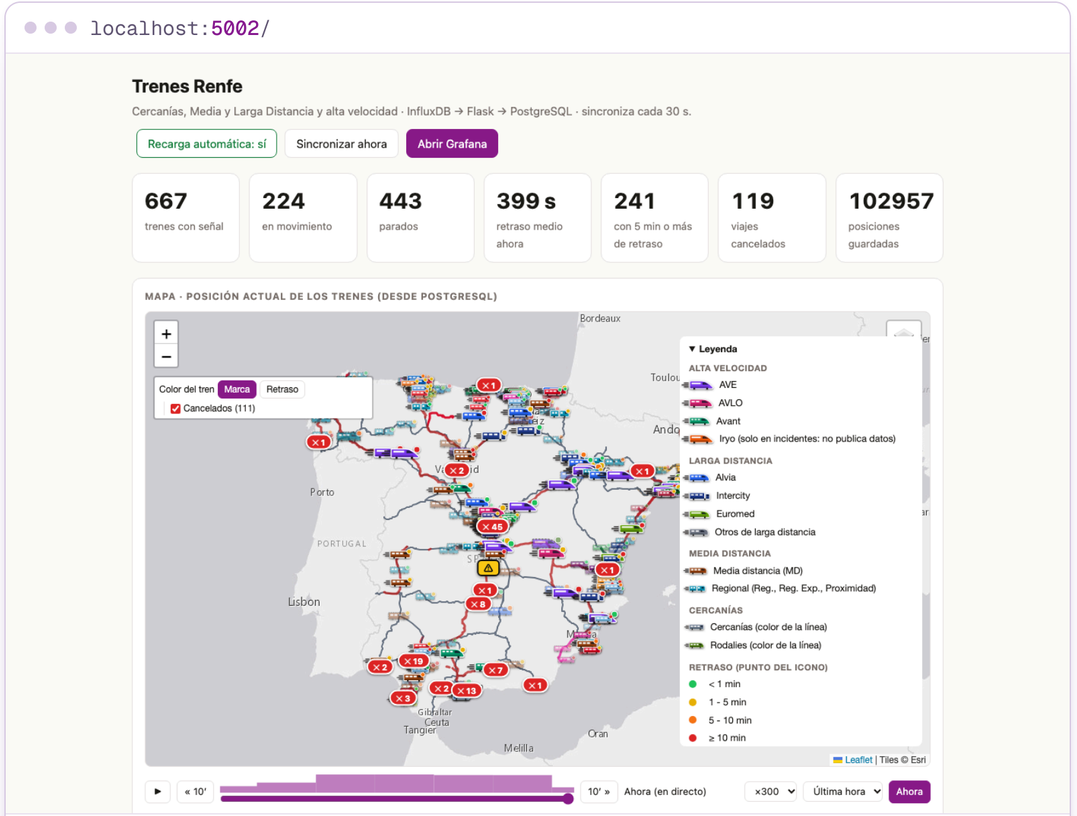
Modo *Marca* o modo *Retraso* (verde, amarillo, naranja ≥ 5 min, rojo ≥ 10 min). La elección se recuerda.

Movimiento

En movimiento: opaco y con estela. Parado: translúcido y borde discontinuo.

Capas y leyenda

Se pueden ocultar vías, marcas, parados, cancelados e incidentes. Etiqueta bajo el icono desde zoom 8.



Un tren se considera parado si Renfe lo marca STOPPED_AT o su posición no cambia más de ~30 m en 3 minutos.

El mapa también sabe volver atrás

Línea de tiempo

- El deslizador recorre el histórico de PostgreSQL (48 h).
- Las barras muestran de cuándo hay datos; los huecos son momentos sin `trenes.py`.
- Reproducir a ×60, ×300 o ×900, saltos de ±10 min y ventana de 1 h, 6 h, 24 h o todo.
- Al pasar al pasado se desactiva la recarga automática.

Viajes cancelados

- Renfe marca `CANCELED` en trip_updates y quita el tren de las posiciones.
- En larga distancia se saca la marca, el número y de dónde a dónde iba.
- Marcador rojo «✕ N» en la estación de origen; popup con la lista.
- Cercanías: solo se cuentan, porque su GTFS no trae paradas por viaje.

Incidentes

- Tabla `incidentes` con sucesos escritos a mano (incluido Iryo, sin datos abiertos).
- Marcador amarillo con aviso, visible solo desde su hora.
- `app.py` siembra el choque de Urda (Toledo) del 28/10/2026.
- Posición aproximada; sin cifras provisionales, se remite a las fuentes.

Un dashboard, dos fuentes de datos, dos filas

«Trenes Renfe» (uid `trenes-renfe`) se carga solo por *provisioning*: datasources y dashboard viven en `grafana/`.

Fila InfluxDB · tiempo real

Trenes circulando, retraso medio, trenes con retraso, velocidad máxima estimada, trenes y retraso por núcleo en el tiempo y mapa de últimas posiciones.

Fila PostgreSQL · agregados

Trenes con señal, en movimiento y parados; retraso por hora y por núcleo; top 10 de líneas con más retraso; puntualidad, mapa y tabla de los más retrasados.

Lo que los datos de Renfe no dicen

Los feeds son abiertos pero limitados. La práctica lo asume en vez de inventar.

Sin velocidad ni rumbo

`vel` se estima entre dos posiciones consecutivas y solo es orientativa.

Posición a saltos

No es un GPS continuo: se queda fija minutos y luego salta.

Sin hora por tren en LD

Se usa la hora de recepción del feed.

Solo Renfe

Iryo y otros operadores privados no publican datos abiertos; solo aparecen en incidentes a mano.

Cancelados sin posición

Renfe quita el tren y deja una entrada `CANCELED`; hay que reconstruir el recorrido con el GTFS.

Dos scripts: arrancar y parar

Los datos se conservan en volúmenes. Para empezar de cero basta con borrarlos.

`api/.cache/` guarda los GTFS descargados (~17 MB) y se renuevan cada semana. Si falta `static/red.geojson`, el mapa funciona igual, sin el trazado de la red.

PRACTICA3

```
$ ./start.sh      # contenedores + Terminal con api/trenes.py
$ ./stop.sh       # para todo, los datos se conservan

# generar el trazado del mapa (una vez)
$ python api/generar_lineas.py   # static/lineas.geojson
$ python api/generar_red.py      # static/red.geojson

# borrar todos los datos
$ docker compose down -v
```

FIN

Con los contenedores arrancados, todo se abre desde aquí.

Flask
Mapa y línea de tiempo

`localhost:5002`

Grafana
Dashboard Trenes Renfe

`localhost:3001`

pgAdmin
Base PostgreSQL

`localhost:5050`

Chronograf
Explorar InfluxDB

`localhost:8888`

InfluxDB
API de la base renfe

`localhost:8086`

Salud
Estado de Flask

`/health`